

Fire Detection Agent

File: FireDetection_Agent_Jorge_Peguro_ITAI1378.ipynb

Jorge Peguero – W214777728

ITAI 1378

This notebook contains the process of building and testing the Fire Detection Agent. It uses the custom model from Yolo_TrainingAndEvaluation_Jorge_Peguro_ITAI1378.ipynb to determine whether the input contains fire or not to send notifications via Telegram.

Environment Setup:

```
%pip install ultralytics opencv-python requests  
#Installing required dependencies
```

Mostrar salida oculta

```
from ultralytics import YOLO  
import cv2  
import requests  
import numpy as np  
from PIL import Image  
import matplotlib.pyplot as plt #Importing required libraries and tools
```

```
from google.colab import drive #Accessing Google Drive
```

The notebook starts by setting up the environment:

- Ultralytics: Necessary to call and run the custom model.
- OpenCV: handles video file reading and frame extraction.
- Requests: used to call the Telegram Bot API and download the sample videos.
- Pillow / matplotlib — used to display detected frames inline within the Colab notebook.

Connecting Google Drive:

```
from google.colab import drive #Accessing Google Drive  
drive.mount('/content/drive')
```

Mounted at /content/drive

The notebook connects the Google Colab environment with Google Drive to access the best.pt file stored from previous training.

Loading the model:

```
model = YOLO('/content/drive/MyDrive/CV_Project/CV_Project/weights/best.pt') #Getting the custom model from Google Drive

class_names = model.names #Getting class names

print(class_names)

{0: 'fire', 1: 'nofire'}
```

The model is loaded from the best.pt file stored in Google Drive. Model.names returns a dictionary with the class index and name.

Downloading Test Videos:

```
video1_nofire = {'url':'https://www.pexels.com/es-es/download/video/33968796/', 'path':'video1_nofire.mp4'} #Defining sample videos
video2_fire = {'url':'https://www.pexels.com/es-es/download/video/4825783/', 'path':'video2_fire.mp4'}
video3_fire = {'url':'https://www.pexels.com/es-es/download/video/29719119/', 'path':'video3_fire.mp4'}

videos = [video1_nofire, video2_fire, video3_fire]

for video in videos: #Downloading sample videos
    response = requests.get(video['url'], stream = True)
    with open(video['path'], 'wb') as f:
        for chunk in response.iter_content(chunk_size=1024*1024):
            if chunk:
                f.write(chunk)
```

Three videos are downloaded from Pexels to test the system in different conditions.

Video1_nofire: There is no presence of fire; the system should not send any alert.

Video2_fire: Clearly presnece of fire and smoke, the system should send an urgent message.

Video3_fire: Visible smoke, the system should send an urgent message.

The videos are downloaded using the request library and stored in the environment with the name defined on path.

Connecting with Telegram API:

```
token = 'token' #Connecting and testing the Telegram Bot
chat_id = 'chat id'
text = 'Fire detected'

request = requests.post(
    f'https://api.telegram.org/bot{token}/sendMessage',
    json={
        'chat_id': chat_id,
        'text': text
    }
)
```

This code block establishes and tests the connection with Telegram API, using the requests library.

Defining Messages:

```
text_1 = 'URGENT: Manual Verification Required - Potential Fire Activity Detected\nOur automated fire monitoring system has identified suspicious activity. The current detection confidence level is 0.5. Please verify the situation immediately.\n\n#Creating the text samples that the Agent is going to use when sending messages'

text_2 = 'CRITICAL PRIORITY: High-Confidence Fire Detection - Emergency Protocols Required\nThe fire detection system has identified a high-probability fire event. This event has reached a critical confidence level of 0.7. Immediate action is required.\n\n#Creating the text samples that the Agent is going to use when sending messages'

text_3 = 'EMERGENCY: Fire Confirmed - Automated Dispatch Initiated\nCRITICAL INCIDENT ALERT - LEVEL: MAXIMUM\nEmergency services and local fire authorities have been automatically notified and are en route to the location. Please evacuate the area immediately.\n\n#Creating the text samples that the Agent is going to use when sending messages'
```

The notebook defines the messages that the agent will send when indicated.

Send_alert Function:

```
def send_alert(confidence): #creating the function that sends messages through the Telegram API based on thresholds
    if confidence < 0.5:
        request_send_alert = requests.post(f'https://api.telegram.org/bot{token}/sendMessage', json={'chat_id': chat_id, 'text': text_1})
    elif 0.5 <= confidence < 0.8:
        request_send_alert = requests.post(f'https://api.telegram.org/bot{token}/sendMessage', json={'chat_id': chat_id, 'text': text_2})
    else:
        request_send_alert = requests.post(f'https://api.telegram.org/bot{token}/sendMessage', json={'chat_id': chat_id, 'text': text_3})
```

Send_alert takes as an argument of confidence, which is the top1 confidence returned by the model in the results object. Based on confidence the agent would send any of the three messages stated in the previous section using the requests library and the Telegram API.

Confidence	Alert
<0.5	Requires human confirmation.
0.5<= n <0.8	Critical Priority
0.8<	EMERGENCY

Predict_video Function:

```
def predict_video(video_path): #Agent that gets the sample videos and calls the model to determine whether there is or not fire on the sample video, it also displays the detected frame and sends an alert if fire is detected
    cap = cv2.VideoCapture(video_path)
    fps = cap.get(cv2.CAP_PROP_FPS)

    frame_count = 0
    fire_count = 0
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break

    #Analyzing a frame per second
    if frame_count % int(fps) == 0:
        results = model.predict(frame)
        class_list = results[0].probs.top1
        confidence = results[0].probs.top1conf

        if class_names[class_list] == 'fire':
            if confidence < 0.5:
                print('Fire detected but I'm not sure')
            elif 0.5 <= confidence < 0.8:
                print('Very sure it is fire')
            else:
                print('Sure it is fire')

    #Displaying the detected frame
    cv2.putText(frame, f'Class: {class_names[class_list]} Confidence: {confidence:.2f}', (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 0), 2)

    img = Image.fromarray(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
    plt.imshow(img)
    plt.axis('off')
    plt.show()

    #Sending the alert
    fire_count += 1
    if fire_count == 5:
        send_alert(confidence)
        fire_count = 0

    frame_count += 1

    cap.release()
```

This is the core function of the notebook. It processes a video frame by frame and applies the YOLO classifier at a rate of 1 frame per second. The function takes as an argument the video path. Step-by-step logic:

- Open the video with `cv2.VideoCapture` and read the FPS.
- Iterate over frames with a while loop.
- Using `frame_count % int(fps) == 0` the function ensures that the model only analyzes one frame per second.
- `Model.predict()` to run inference.
- Extract results
- If fire is detected, print the confidence level, overlay the prediction text on the frame, and display the frame in the notebook using `matplotlib`.
- Count consecutive fire detections with `fire_count`. Once 5 fire detections have accumulated, `send_alert()` is called and the counter resets.

The number of 5 fire detections is arbitrary; it is just a way to ensure that a false positive does not result in sending a message; the system requires a minimum of fire detections to send the alert.